Unit II

OOPs

2.1 What is a class in Java?

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

Java is an object-oriented programming language. Everything in Java is associated with classes and objects, along with its attributes and methods. Class is declared using class keyword. A class contain both data and code that operate on that data. The data or variables defined within a class are called instance variables and the code that operates on this data is known as methods. Thus, the instance variables and methods are known as class members. Class is also known as a user defined datatype.

Syntax to declare a class:

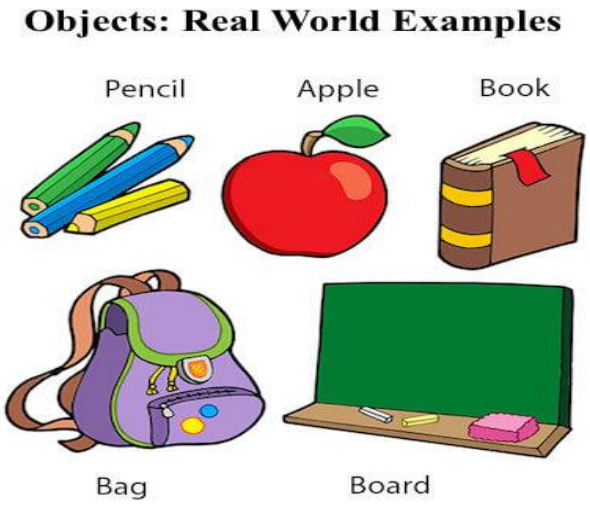
```
class classname
{
type instance-variable;

    return type instance-method (parameter-list)
    {
        //body of method
    }
}
```

Explanation: - Class is declared by Keyword class.

Class-name indicates the name of class. Name of class is user defined name & used as user defined data type of its variable (object). – In a class we can declare variables as well as functions.

Objects in Java

In object-oriented programming, we design a program using objects and classes. An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.



An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

An object has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

For Example, Pen is an object. Its name is cello; color is blue, known as its state. It is used to write, so writing is its behavior.

An object is an instance of a class. A class is a template or blueprint from which objects are created. So, an object is the instance (result) of a class.

Object Definitions:

- An object is *a real-world entity*.
- An object is *a runtime entity*.

- The object is *an entity which has state and behavior*.
- The object is *an instance of a class*.

Object and Class Example: main within the class

We have created a Stud class which has two data members' rollno and name. We are creating the object of the Stud class by new keyword and printing the values through object.

Here, we are creating a main ( ) method inside the class.

File: Student.java

```
//Java Program to illustrate how to define a class and fields

public class Stud {                                  //defining fields
 int rollno;                                         //field or data member or instance variable
 String name;                                        //creating main method inside the class
 public static void main( String args[ ] ){          //Creating an object or instance
  Stud s1=new Stud( );                          //creating an object of Student
  //Printing values of the object
  System.out.println(s1.rollno);                //accessing member through reference variable
  System.out.println(s1.name);
 } }
```

Output:

```
0
null
```

Multiple Objects – You can create multiple objects of one class: It is also possible to create two or more references to the same object .Both B1and B2 refer to the same object.

Example - Java Program to create two objects of a class

File: Box.java

```
public class Box
{
int x=5;
public static void main(String args[ ])
{
Box B1 = new Box( );            //Object1
Box B2 = new Box( );            //Object2

System.out.println(B1.x);
System.out.println(B2.x);
}
}
```

o/p:

5
5

Another example to create main( ) method outside the class

File: test.java

```
class box1{
double height;                                  //field or data member or instance variable
double width;
double depth;
void gethwd(double h,double w,double d){        //method or instance method
height=h;
width=w;
depth=d;
}
void volume( ){                                 //method or instance method
System.out.println("volume is " + (height * width * depth) );
}
}
//creating main method outside the class
class test{
public static void main( String args[ ] ){
box1 b1=new box1( );                            //creating an object
b1.gethwd(10,20,30);                            //accessing instance method with object
b1.volume( );                                   //accessing instance method with object
  }  }
```

Visibility Controls/Access Modifiers:

The variables and methods of a class are visible everywhere in the program. However, sometimes we may want them not to be accessible outside of the class. We can achieve this in Java by applying visibility modifiers to instance variables and methods.

The visibility modifiers are also known as access modifiers. Access modifiers determine the accessibility of the members of a class. Java provides three types of visibility modifiers**: public, private and protected**

They provide different levels of protection as described below.

Public Access: in public mode, a variable or method is visible to the entire class in which it is defined. A variable or method declared as public has the widest possible visibility and accessible everywhere.

Private Access: private fields have the highest degree of protection. They are accessible to their own class only. They cannot be inherited by subclasses and therefore not accessible in subclasses. In the case of overriding, public methods cannot be redefined as private type.

Protected Access: The visibility level of a "protected" field lies in between the public access and friendly access. That is, the protected modifier makes the fields visible not only to all classes and subclasses in the same package but also to subclasses in other packages

(No modifier) (Default): When no access modifier is specified, the member defaults to a limited version of public accessibility known as "friendly" level of access.

The difference between the "public" access and the "friendly" access is that the public modifier makes fields visible in all classes, regardless of their packages while the friendly access makes fields visible only in the same package, but not in other packages.

Private protected Access: A field can be declared with two keywords private and protected together. This gives a visibility level in between the "protected" access and "private" access. This modifier makes the fields visible in all subclasses regardless of what package they are in. Remember, these fields are not accessible by other classes in the same package.

2.2 Constructors and static members

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Every time an object is created using the new() keyword, at least one constructor is called.

It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.

There are two types of constructors in Java: no-arg constructor, and parameterized constructor.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type

3. A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

Syntax of default constructor:

<class_name>(){}

Example:

```
1. //Java Program to create and call a default constructor
2. class Bike1{
3. //creating a default constructor
4. Bike1(){System.out.println("Bike is created");}
5. //main method
6. public static void main(String args[]){
7. //calling a default constructor
8. Bike1 b=new Bike1();
9. }
10. }
```

Output

Bike is created

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

In this example, we have created the constructor of Student class that has two parameters. We can have any number of parameters in the constructor.

```
1. //Java Program to demonstrate the use of the parameterized constructor.
2.    class Student4{
3.     int id;
4.     String name;
5.     //creating a parameterized constructor
6.     Student4(int i,String n){
7.     id = i;
8.     name = n;
9.     }
10.    //method to display the values
11.    void display(){System.out.println(id+" "+name);}  }
12.   Class paraTest{
13.    public static void main(String args[]){
14.    //creating objects and passing values
15.    Student4 s1 = new Student4(111,"Karan");
16.    Student4 s2 = new Student4(222,"Aryan");
17.    //calling method to display the values of object
18.    s1.display();
19.    s2.display();
20.   }
21.   }
```

Output
222 Aryan

111Karan

Constructor Overloading in Java

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example of Constructor Overloading

```
1. //Java program to overload constructors
2. class Student5{
3.     int id;
4.     String name;
5.     int age;
6.     //creating two arg constructor
7.     Student5(int i,String n){
8.     id = i;
9.     name = n;
10.    }
11.    //creating three arg constructor
12.    Student5(int i,String n,int a){
13.    id = i;
14.    name = n;
15.    age=a;
16.    }
17.    void display(){System.out.println(id+" "+name+" "+age);}
18.    class test3{
19.    public static void main(String args[]){
20.    Student5 s1 = new Student5(111,"Karan");
21.    Student5 s2 = new Student5(222,"Aryan",25);
22.    s1.display();
23.    s2.display();
24.   }
25. }
```

Output

```
111 Karan 0
222 Aryan 25
```

2.2 Static keyword:

The static keyword in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

The static can be:

- Variable (also known as a class variable)
- Method (also known as a class method)
- Block
- Nested class

If you declare any variable as static, it is known as a static variable.

1) Java static variable

The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.

The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes your program memory efficient (i.e., it saves memory).

```
class Student{
    int rollno;
    String name;
    String college="ITS";
}
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

```
//Java Program to demonstrate the use of static variable
class Student{
  int rollno;//instance variable
```

```
  String name;
  static String college ="COCSIT";              //static variable

  Student(int r, String n){                 //constructor
  rollno = r;
  name = n;
  }
  //method to display the values
  void display (){System.out.println(rollno+" "+name+" "+college);}
}
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
 s1.display();
 s2.display();
 }
}
```

Output:

```
111 Karan ITS
222 Aryan ITS
```

2) Java static method

- If you apply static keyword with any method, it is known as static method.
- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Example of static method

```
//Java Program to demonstrate the use of a static method.
class Student{
    int rollno;
    String name;
    static String college = "SHARDA";
    //static method to change the value of static variable
    static void change(){
    college = "COCSIT";
```

```
    }
    //constructor to initialize the variable
    Student(int r, String n){
    rollno = r;
    name = n;
    }
    void display(){System.out.println(rollno+" "+name+" "+college);}
}
public class TestStaticMethod{
   public static void main(String args[]){
   Student.change();//calling change method
   //creating objects
   Student s1 = new Student(11,"Kiran");
   Student s2 = new Student(12,"Karan");
   Student s3 = new Student(13,"Komal");
   //calling display method
   s1.display();
   s2.display();
   s3.display();
   }
}
```

Output:

```
        11 Kiran COCSIT
        12 karan COCSIT
        13 Komal  COCSIT
```

Another example of a static method that performs a normal calculation

```
//Java Program to get the cube of a given number using the static method
  class Calculate{
  static int cube(int x){
  return x*x*x;
  }
    public static void main(String args[]){
  int result=Calculate.cube(5);
  System.out.println(result);
  }
}
```

Output:

125

2) Java static block

- Is used to initialize the static data member.
- It is executed before the main method at the time of classloading.

Example of static block

```
class A2{
 static { System.out.println("static block is invoked"); }
  public static void main(String args[]){
  System.out.println("Hello main");
  }
}
```

Output:

static block is invoked

Hello main

2.3 Encapsulation, Inheritance, this and super keyword

Encapsulation

Encapsulation is one of the **four fundamental Object-Oriented Programming (OOP) principles**, along with Inheritance, Polymorphism, and Abstraction.

Definition: Encapsulation is the **wrapping of data (variables)** and **code (methods)** together into a **single unit** (a class) and restricting direct access to some of the object's components.

- Technically, in encapsulation, the variables or the data in a class is hidden from any other class and can be accessed only through any member function of the class in which they are declared.
- In encapsulation, the data in a class is hidden from other classes, which is similar to what **data-hiding** does. So, the terms "encapsulation" and "data-hiding" are used interchangeably.
- Encapsulation can be achieved by declaring all the variables in a class as private and writing public methods in the class to set and get the values of the variables.

```
class Employee {
   // Private fields (encapsulated data)
```

```
    private int id;
    private String name;
    // Setter methods
    public void setId(int id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    // Getter methods
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
}
public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee();

        // Using setters
        emp.setId(101);
        emp.setName("Geek");
        // Using getters
        System.out.println("Employee ID: " + emp.getId());
        System.out.println("Employee Name: " + emp.getName());
    }
}
```

Output -

```
Employee ID: 101
Employee Name: Geek
```

Inheritance

Inheritance is an important pillar of OOP (Object-Oriented Programming). It is the mechanism in java by which one class is allowed to inherit the features (fields and methods) of another class. In inheritance, one object acquires all the properties and behaviors of a parent object.

Inheritance Terminologies:

- **Super Class:** The class whose features are inherited is known as superclass (or a base class or a parent class).
- **Sub Class:** The class that inherits the other class is known as a subclass (or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- **Reusability:** Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we need then in this situation, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of an existing class.

The keyword used for inheritance is **extends**.

Syntax :

```
   class derived-class extends base-class
{
  //methods and fields
}
```

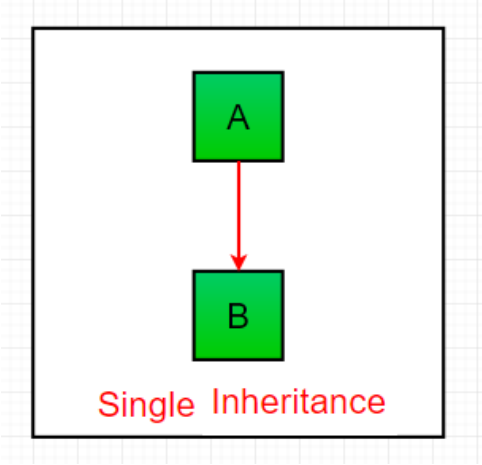
Example:

```
    class Emp{
          float salary=40000;
         }
         class Prg extends Emp{
          int bonus=10000;
          public static void main(String args[]){
            Prg p=new Prg ();
            System.out.println("Programmer salary is:"+p.salary);
            System.out.println("Bonus of Programmer is:"+p.bonus);  }}
```

Types of Inheritance in Java

Following are the different types of inheritance in Java.

1. Single Inheritance: When a class inherits another class, it is known as a single inheritance. In single inheritance, subclasses inherit the features of one superclass. In the image below, class A serves as a base class for the derived class B.



// Java program to illustrate the concept of single inheritance

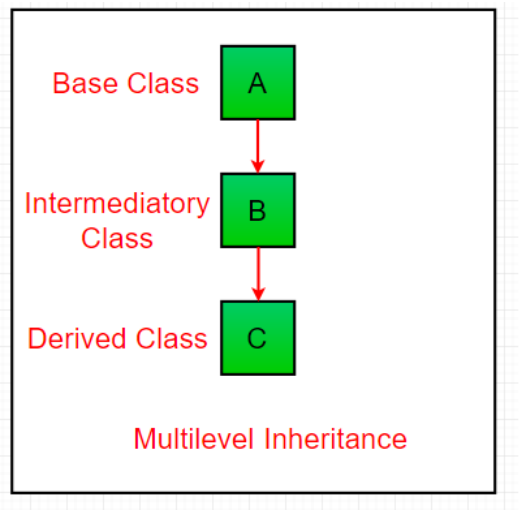
```
class Student{
void name(){System.out.println("Name is Amol...");}
}
class Id extends Student{
void rollno(){System.out.println("Roll no is 25");}
}
class SingleTest{
public static void main(String args[]){
Id d=new Id();
d.rollno();
d.name();
}
}
```

Output

Roll no is 25

Name is Amol...

2. Multilevel Inheritance: In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the



// Java program to illustrate concept of Multilevel inheritance

```
class one {
   public void print_green() {
      System.out.println("Green");
         } }
class two extends one {
   public void print_red() { System.out.println("Red");
         } }
class three extends two {
   public void print_blue()    {
      System.out.println("Blue");
         } }
public class MultiTest {
   public static void main(String[] args)     {
      three C = new three();
      C.print_blue();
      C.print_red();
      C.print_green();
         } }
```

Output
Blue
Red
Green

3. Hierarchical Inheritance: When two or more classes inherit a single class, it is known as hierarchical inheritance. In the below image, class A serves as a base class for the derived class B, C and D.

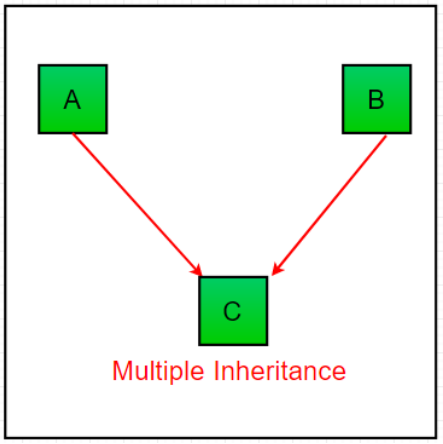
// Java program to illustrate the concept of Hierarchical inheritance

```
class A {
   public void printA( ) { System.out.println("This is Class A"); }
}

class B extends A {
   public void printB( ) { System.out.println("This is Class B"); }
}

class C extends A {
   public void printC( ) { System.out.println("This is Class C"); }
}

class D extends A {
   public void printD( ) { System.out.println("This is Class D"); }
}
public class hierarTest {
   public static void main(String[] args)    {
      B  Bobj = new B( );
      Bobj.printA( );
      Bobj.printB( );

      C Cobj = new C( );
      Cobj.printA( );
      Cobj.printC( );

      D Dobj = new D();
      Dobj.printA( );
      Dobj.printD( );

   } }
```

Output

This is Class A
This is Class B
This is Class A
This is Class C
This is Class A
This is Class D

4. Multiple Inheritance : To reduce the complexity and simplify the language, multiple inheritances is not supported in java. Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class. So whether you have same method or different, there will be compile time error.

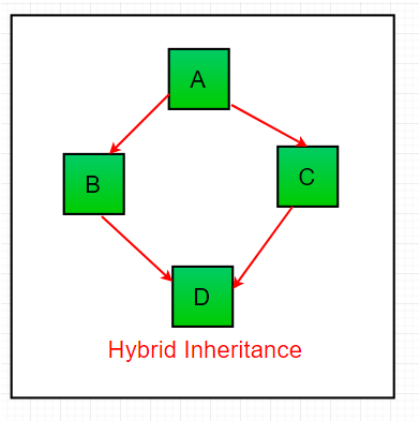


// Java program to illustrate the concept of Hierarchical inheritance

```
class A{
void msg( ){System.out.println("Hello");}
}
class B{
void msg( ){System.out.println("Welcome");}
}
class C extends A,B{     //suppose if it were
public static void main(String args[]){
  C obj=new C( );
  obj.msg( );     //Now which msg() method would be invoked?
} }
```

Output
Compile Time Error

5. Hybrid Inheritance (Through Interfaces): It is a mix of two or more of the above types of inheritance. Since java doesn't support multiple inheritances with classes, hybrid inheritance is also not possible with classes. In java, we can achieve hybrid inheritance only through Interfaces.



this keyword

There can be a lot of usage of **java this keyword**. These are given below.

1. this can be used to refer current class instance variable.
2. this can be used to invoke current class method (implicitly)
3. this () can be used to invoke current class constructor.
4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructor call.
6. this can be used to return the current class instance from the method.

1) Program without using this keyword

```
class thisEx{
int rollno;
String name;
float fee;
thisEx(int rollno,String name,float fee){
rollno=rollno;
name=name;
fee=fee;
}
void display(){
```

```
System.out.println(rollno+" "+name+" "+fee);}
}
class TestThis{
public static void main(String args[]){
thisEx s1=new thisEx(111,"ankit",5000f);
thisEx s2=new thisEx(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

```
0 null 0.0
0 null 0.0
```

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

1) **this: to invoke current class instance variable.**
2) **this keyword to solve above problem**

```
class thisEx1{
int rollno;
String name;
float fee;
thisEx1(int rollno, String name, float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){
System.out.println(rollno+" "+name+" "+fee);}
}
class TestThis{
public static void main(String args[]){
thisEx1 s1=new thisEx1(111,"ankit",5000);
thisEx1 s2=new thisEx1(112,"sumit",6000);
s1.display();
s2.display();
}}
```

Output:

```
111 ankit 5000
112 sumit 6000
```

Remember that this keyword is used only when names of formal argument and instance variable is same. If local variable (formal arguments) and instance variable names are different, then there is no need to use this keyword like above program.

2) this: to invoke current class method

You may invoke the method of the current class by using this keyword. If you don't use this keyword, compiler automatically adds this keyword while invoking the method. Let's see the example.

```
class A{
void m(){
System.out.println("hello m");}
void n(){
System.out.println("hello n");
this.m();
}  }
class TestThis4{
public static void main(String args[]){
A a=new A();
a.n();
}}
```

Output:
hello n
hello m

this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Calling default constructor from parameterized constructor:

```
class A{
A(){ System.out.println("hello a");}
A(int x){
this();
System.out.println(x);
}
}
class TestThis5{
public static void main(String args[]){
```

```
A a=new A(10);
}}
 Output:
hello a
10
```

Super keyword

The super keyword in Java is a reference variable used to refer to the **immediate parent class** object.

It is mainly used for:

1. Accessing **parent class methods**.
2. Accessing **parent class constructors**.
3. Accessing **parent class fields (variables)**.

◆ 1. Accessing Parent Class Method

If a subclass has overridden a method of its parent class, we can call the parent class version using super.methodName().

```
java
CopyEdit
class Animal {
   void sound() {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal {
   void sound() {
      super.sound(); // calls Animal's sound()
      System.out.println("Dog barks");
   }
}

class Test {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.sound();
   }
}
```

Output:

```
css
```

CopyEdit
Animal makes a sound
Dog barks

◆ 2. Accessing Parent Class Variable

If both parent and child have same variable name, `super` helps to access the parent variable.

java
CopyEdit
```
class Parent {
   int x = 50;
}

class Child extends Parent {
   int x = 100;

   void show() {
      System.out.println("Child x = " + x);
      System.out.println("Parent x = " + super.x);
   }
}

class Test {
   public static void main(String[] args) {
      Child c = new Child();
      c.show();
   }
}
```

Output:

java
CopyEdit
```
Child x = 100
Parent x = 50
```

◆ 3. Calling Parent Class Constructor

You can call the parent class constructor using `super()` from the child class constructor. This must be the **first statement** in the constructor.

java
CopyEdit
```
class Animal {
   Animal() {
      System.out.println("Animal constructor");
   }
}
```

```
class Dog extends Animal {
   Dog() {
      super(); // calls Animal()
      System.out.println("Dog constructor");
   }
}

class Test {
   public static void main(String[] args) {
      Dog d = new Dog();
   }
}
```

Output:

```
kotlin
CopyEdit
Animal constructor
Dog constructor
```

2.4 Polymorphism

The word polymorphism means "many forms".

In Java, polymorphism allows the same method (or object) to perform different actions based on the situation. It refers to the ability of object-oriented programming languages t**o differentiate between entities with the same name efficiently**.

Types of Polymorphism

Polymorphism in Java is mainly of 2 types as mentioned below:

1. Method Overloading
2. Method Overriding

1. Method Overloading

If a class has multiple methods having same name but different parameters, then it is known as **Method Overloading**.

Suppose you have to perform addition of any number of arguments, then in this situation the concept of method overloading helps. Method overloading increases the readability of the program.

There are two ways to overload the method in java.

- By changing number of arguments

- By changing the data type

Method Overloading: By changing number of arguments

In this example, we have created two methods named add( ). First add( ) performs addition of two numbers and second add( ) performs addition of three numbers.

In this example, we are creating static methods so that we don't need to create instance for calling methods.

```
class Adder{
static int add(int a,int b){return a+b;}
static int add(int a,int b,int c){return a+b+c;} }
class Test1{
public static void main(String args[ ]){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(20 ,30,40));
}}
```

Output:

```
22
90
```

2) Method Overloading: changing data type of arguments

In this example, we have created two methods that differs in data type. The first add method receives two integer arguments and second add method receives two double arguments.

```
class Adder{
static int add(int a, int b){return a+b;}
static double add(double a, double b){return a+b;}
}
class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}}
```

Output:

```
22
24.9
```

2. Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.

In other words, if a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

- The method must have the same name as in the parent class
- The method must have the same parameter as in the parent class.
- There must be an IS-A relationship (inheritance).

Example of method overriding

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

```
//Java Program to illustrate the use of Java Method Overriding
Class Vehicle{
  void run(){System.out.println("Vehicle is running");}
}
//Creating a child class
class Bike2 extends Vehicle{
  //defining the same method as in the parent class
  void run() { System.out.println("Bike is running safely");}
  public static void main(String args[]){
  Bike2 obj = new Bike2();//creating object
  obj.run();//calling method
  }  }
```

Output:

Bike is running safely

Another example of method Overriding

```
//Java Program to demonstrate Java Method Overriding
//where three classes are overriding the method of a parent class.

class Bank{
int getROI(){return 0;}
}
//Creating child classes.
class SBI extends Bank{
int getROI (){return 8;}
}


class ICICI extends Bank{
int getROI (){return 7;}
}
class AXIS extends Bank{
int getROI (){return 9;}
}
//Test class to create objects and call the methods
class Test2{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s. getROI ( ));
System.out.println("ICICI Rate of Interest: "+i. getROI( ));
System.out.println("AXIS Rate of Interest: "+a. getROI ( ));
}
}
```

Output:

SBI Rate of Interest: 8

ICICI Rate of Interest: 7

AXIS Rate of Interest: 9

2.5 Garbage Collection

Garbage collection in Java is an **automatic memory management process** that helps Java programs run efficiently. Java programs compile to bytecode that can be run on a Java Virtual Machine (JVM). When Java programs run on the JVM, objects are created in the heap memory.

- **Heap memory** is the part of Java memory (inside JVM) used for **dynamic memory allocation**.
- Whenever we create objects (using `new` keyword), they are stored in the **heap**.
- Heap is **shared** among all threads of a Java application.

Eventually, some objects will no longer be needed. The garbage collector finds these unused objects and deletes them to free up memory.

Java garbage collector automatically **identifies and removes unused objects**, freeing up memory in the heap. It runs in the background as a daemon thread, helping to manage memory efficiently without requiring the programmer's constant attention.

Working of Garbage Collection

- Java garbage collection is an automatic process that manages memory in the heap.
- It identifies which objects are still in use (referenced) and which are not in use (unreferenced).
- It removes the objects that are unreachable (no longer referenced).
- The programmer does not need to mark objects to be deleted explicitly. Garbage collection is implemented within the JVM.

***